

8th International Conference on Industry of the Future and Smart Manufacturing

ROS2SmolVLA: Enabling Small Vision-Language-Action Models for Integration into Industrial-Grade Lightweight Robots

Nils Mandischer^{a,1,*}, Noah Böckmann^{a,1}, Ludwig Holl^a, Lars Mikelsons^a

^aUniversity of Augsburg, Chair of Mechatronics, Am Technologiezentrum 8, 86159 Augsburg, Germany

Abstract

Industrial demand changes the paradigms of production. Due to smaller batch sizes and more variations in products, companies face a growing challenge to adopt more adaptive production systems. In particular, robot-based automation is usually static and fails to respond to constantly changing processes. Vision-Language-Action (VLA) Models are a promising opportunity to mitigate this challenge by generating robot actions based on the observed system state. However, current research either focuses on large models that cannot be computed on premise, creating compliance and security challenges, or use lab-grade robot hardware that obscures exploitation in real industrial settings. In this work, we adapt Hugging Face's SmolVLA for Universal Robots lightweight robots. Further, we release the open-source repository ROS2SmolVLA that implements an interface for ROS 2 to SmolVLA, and makes it applicable for industrial-grade hardware. By this, we allow a lenient adoption into lab and industrial environments. We validate the functionality of SmolVLA for a Universal Robots UR10e using a pick-and-place task and give implementation guidelines. Our findings support that SmolVLA is a well-suited option for small-sized tasks that need to be computed on premise.

© 2026 The Authors. Check in the contract: Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the 8th International Conference on Industry of the Future and Smart Manufacturing.

Keywords: vision-language-action model; VLA; adaptive production; human-robot interaction; lightweight robots; collaborative robots

1. Introduction

Modern industries face a paradigm change from rigid and sequential towards more flexible and adaptive production systems. These go hand in hand with smaller batch sizes, more variety in products, and move away from scheduled to on-demand production. While established automation systems work well in structured environments, flexible and adaptive production systems lead to more economic risks. In fact, a robot system that originally operated continually on a single work process must now be available for many processes and must adapt its execution to already minor changes in products. In recent years, automation systems met this demand by establishing more adaptivity in planning and execution. Flexible Manufacturing Systems (FMS) provide more variety to changes in task sequence and batch

* Corresponding author. Tel.: +49 821 598 69287

¹ Both authors contributed equally to this work.

E-mail address: nils.mandischer@uni-a.de

sizes by adding more general purpose automation systems. In contrast, Reconfigurable Manufacturing Systems (RMS) use less hardware with a larger range of functionality and automation capabilities, leading to a lowered risk of over-automation. In recent years, Artificial Intelligence (AI) gained widespread use over the entire product life cycle. AI provides solutions for virtually all core concepts of RMS: modularity, integrability, convertibility, diagnosability, customization, and scalability [1, 2]. Thus RMS principles were extended into Smart Reconfigurable Manufacturing (SRM) [3]. SRM leverages the opportunities of AI capabilities in reconfigurable manufacturing, allowing for more adaptive planning and execution, and better abilities to react to unforeseen changes in the factory environment.

Reinforcement Learning (RL) has become a primary tool for SRM as it allows to handle high-dimensional data subject to uncertainty [4]. RL gained traction with the introduction of digital twins into manufacturing, moving many planning and optimization operations from real-world into virtual environments. Recently, approaches like transformer-based RL using self-attention extended the abilities of production agents towards long-term temporal reasoning [5]. With the quick rise of Large Language Models (LLMs), large-scale transformer models also got introduced into SRM [6]. LLMs allow reasoning on big data without requiring trial-and-error testing like RL methods. Recently, transformer-based Vision-Language-Action models (VLAs) emerged as a means of adaptively controlling robots [7, 8]. VLAs analyze the status of the workspace through sensor-based observation and Vision-Language Models (VLMs). The tokenized results are then used by an Action Expert to generate robot control commands in real-time. VLAs are commonly trained on teleoperated demonstrations comprising of sensor data and robot control inputs. LLMs and VLAs rely extensively on external compute and/or proprietary models, raising compliance and security concerns. Small VLAs, like Hugging Face’s SmolVLA [9], that feature significantly less parameters aim to relax these concerns by making LLMs available on premise or on the edge. However, their comparatively small context window prevents generalization about whole factory floors. Instead, these models must fulfill specific and bounded goals and actions. Small VLAs are in the early stages of research and have not been made broadly available to real-world production systems. Instead, research relies on small-scale lab experiments with limited workspace dimensions and payloads.

In this work, we propose ROS2SmolVLA, a ROS 2 interface for SmolVLA that bridges the gap from lab experiments to real industrial-grade hardware. We make SmolVLA available for execution directly on Universal Robots lightweight robots with inference provided by edge devices and consumer-grade hardware comparable to edge PCs. We validate the applicability on an Universal Robots UR10e within a pick-and-place task, and indicate lessons learned from the deployment of SmolVLA in drastically larger workspaces.

The paper is structured as follows: First, related work on VLAs is discussed (Section 2), including a detailed description of Hugging Face’s SmolVLA (Section 2.2). In Section 3, we present our open-source project ROS2SmolVLA that bridges ROS 2 and SmolVLA. Software and data are available via Github and Hugging Face. Afterwards, we present a profound validation of ROS2SmolVLA with an Universal Robotis UR10e in a pick-and-place task (Section 4). Finally, Section 5 summarizes the paper and gives an outlook.

2. Related Work

The usage of VLAs for robotics is a relatively new field with high traction. VLAs are a core innovation for humanoid robots, allowing non-deterministic, adaptive behavior in diverse scenarios. However, VLAs are also finding their way into traditional serial kinematics [7]. Early iterations of language-conditioned policies, such as RT-1 [10], relied on routing independently tokenized embeddings from efficient convolutional or vision transformer networks through standard transformer blocks to output discretized control commands. While effective within structured domains, these architectures exhibited limited open-vocabulary generalization.

2.1. Vision-Language-Action Models and Their Usage in Industrial Environments

The state of the art shifted significantly with the introduction of models like RT-2 [8] and the Open X-Embodiment initiative [11]. These architectures directly co-train VLM backbones (such as PaLI-X [12] or PaLM-E [13]) on web-scale VLM tasks and physical robotic trajectories. By phrasing robotic action generation as a token-prediction problem, these models naturally inherit semantic reasoning capabilities. For instance, given a scene with novel objects and a natural-language instruction, the VLA leverages its knowledge to identify the object before mapping the spatial coordinates to end-effector actions via its embedded policy [8, 11]. Building on this paradigm, OpenVLA [14]

introduced an open-source 7B parameter generalist policy trained natively on the Open X-Embodiment dataset. By fusing open-weight vision-language backbones with robust action tokenization, it democratized access to state-of-the-art cross-embodiment control. Cross-embodiment control refers to single policies enabling the control of diverse robot kinematics (embodiments), overcoming the limitations of traditional control engineering that has to be designed specifically for a kinematic structure [15]. Octo [16], also trained on X-Embodiment, couples a block-wise masked attention pattern, that allows flexibly adding or removing observations during finetuning, with a lightweight diffusion-based action head. By this, Octo allows lenient adaptation towards cross-embodiments and different robot/sensor setups. With 27M (Octo-Small) or 93M (Octo-Base) parameters, finetuning and inference are computable on consumer-grade hardware. Notably, Ghosh et al. [16] report comparable results to much larger models like RT-2-X (55B) [11].

While VLAs demonstrate remarkable scaling in laboratory settings, deploying them within high-throughput production environments introduces severe constraints. First, standard automation systems demand fast control frequencies. Particularly in VLAs with many billion parameters, inference may induce latencies above an acceptable threshold. On the one hand, current research overcomes this limitation through aggressive quantization, tensor parallelism, and action-chunking mechanisms where the model outputs a sequence of future actions in a single inference step [17, 18]. On the other hand, VLAs with fewer parameters may be used that lead to less latency through faster inference. For instance, MiniVLA [19], a smaller variant of OpenVLA, scales down the large footprint to just 1B parameters by pairing a compact Qwen language backbone with vector-quantized action chunking. Complementing this trend, Hugging Face introduced SmolVLA [9], a highly lightweight 450M-parameter foundation model optimized for deployment on consumer-grade hardware and CPUs. Similarly, small models like PokeVLA [20] demonstrate that optimizing world-knowledge guidance and visual tokens can effectively offset a reduced language-model capacity, allowing high-frequency real-time execution without sacrificing generalization.

Second, most VLAs are highly demanding on hardware, particularly GPUs that are not necessarily available on the edge. Outsourcing compute to proprietary cloud services raises compliance and privacy issues. On the one hand, industrial adopters may decide to use smaller VLAs that are less demanding on compute and can be computed with edge devices. However, smaller models offer less semantic generalization capabilities and require more specialized deployment [9]. On the other hand, instead of fully training the model, only the action head may be fine-tuned while keeping the backbone frozen. This reduces the demand on compute for model training significantly [7]. In addition, Low-Rank Adaptation (LoRA) [21] can be employed to insert parameter-efficient adapters into the attention and multi-layer perceptron layers of the pre-trained vision-language backbone [14, 22]. By training only a fraction of the total parameters, this method lowers VRAM overhead significantly to allow multi-billion-parameter VLA optimization on consumer-grade hardware while preserving the backbone’s crucial zero-shot semantic capabilities.

Third, smaller VLAs are mostly tested and benchmarked in labs with simple and small hardware, like SO-101 or OpenMANIPULATOR. Particularly, SO-101 has been shown to be a suitable test bench for VLAs and robot learning [23, 24]. However, this obscures the usage of the benchmarked VLAs with real industrial-grade hardware as control loops, safety requirements, payloads, and workspace dimensions vary significantly.

In conclusion, VLAs are a promising method for generalization and democratization of robot learning that are at the border of widespread adoption. However, the high latency and compliance concerns related to cloud-based training and inference should move adopters to models with fewer parameters that are computable on-premise and on the edge. These smaller VLAs have not yet been comprehensively benchmarked in industrial settings under the constraints of a real production environment. We aim to close this gap by allowing SmolVLA to be deployed on industrial-grade lightweight robots and by indicating learnings from moving away from purely lab- or consumer-grade hardware.

2.2. SmolVLA

SmolVLA by Shukor et al. [9] is a lightweight VLA model (450M), designed explicitly to democratize VLA training and making it more accessible. The core innovation of SmolVLA lies in its computational efficiency, enabling on-premise execution and down-stream fine-tuning on consumer-grade hardware. The structural backbone of SmolVLA is the pre-trained VLM SmolVLM-2 [25], which is natively optimized for multi-image and sequential video inputs. By incorporating diverse efficiency mechanisms – most notably token-shuffling – its vision encoder reduces redundant spatial representations. For multi-modal fusion, sequential video frames, natural language task descriptions, and robot sensorimotor states are tokenized into a unified discrete embedding space. Afterwards, they are concatenated sequentially into a single multi-modal context window. To enforce real-time inference, the tokenization of input videos

in SmolVLA is strictly bounded to 64 visual tokens per frame. Furthermore, the Action Expert is decoupled from the final layers of the network. By implementing a layer-skipping protocol, the architecture establishes an adjustable tuning parameter that explicitly balances between task performance (fewer skips) and computational efficiency (more skips). Layer-skipping attends the Action Expert only on the first N layers of the VLM backbone. Shukor et al. [9] observe that the later layers provide significantly less features and may be optimized similar to Pareto frontiers.

Departing from conventional VLA architectures that rely uniformly on full self-attention or homogeneous cross-attention mechanisms across all layers, SmolVLA introduces an interleaved attention topology. In this configuration, individual Transformer layers alternate between self- and cross-attention blocks, minimizing computational complexity while retaining global contextual awareness. Finally, the downstream action vector generation is formulated via a Flow Matching Transformer architecture, allowing the policy to model complex, multi-modal action distributions through continuous-time normalizing flows rather than discrete tokenization or mean-squared-error regression. SmolVLA is pre-trained on community-driven trajectories aggregated under the *lerobot* label on Hugging Face. At submission, this repository comprised 1,270 heterogeneous datasets spanning a wide variety of robot embodiments, manipulation tasks, and environmental conditions [9]. To ingest these diverse data, the training pipeline homogenizes proprioceptive dimensions and action spaces into a standardized coordinate framework. This extensive, crowd-sourced multi-task dataset ensures that despite its compact 450M parameters, SmolVLA retains strong out-of-distribution generalization capabilities and robust zero-shot transfer performance across novel environments.

3. ROS2SmolVLA: Adapting SmolVLA for Industrial-Grade Lightweight Robots

In an effort to enable industrial-grade lightweight robots to use SmolVLA, we propose ROS2SmolVLA, a ROS 2-based software interface, trained and validated with data of a Universal Robots UR10e. While the UR10e functions as a validation experiment, the methods are generalizable allowing cross-embodiment control partially without re-training the model. ROS2SmolVLA draws from Hugging Face’s LeRobot framework [26, 27] and the Cartesian Motion Controller by Scherzinger, Roennau, and Dillmann [28]. In addition, the Robotiq Hand-E driver by the Center of Excellence in Artificial Intelligence (University of Krakow) [29] is used. ROS2SmolVLA uses following self-developed and self-recorded resources:

- [ros2smolvla_docker](#): Docker container for the integrated hardware-software setup.
- [ros2smolvla_ur10e_sim/real](#): Execution on the robot featuring a virtual (*sim*) and a real twin (*real*).
- [ros2smolvla_interface_camera](#): ROS 2 camera integration for LeRobot.
- [ros2smolvla_interface_lerobot](#): ROS 2 interface for LeRobot/SmolVLA.
- Hugging Face data and model repository including [training](#) and [validation](#) data, and the [model](#) [30].

All resources are accessible through the docker repository [31] and listed on a central Github webpage which we recommend as an entry point: <https://una-auxme.github.io/en/projects/ros2smolvla/>

3.1. System Architecture and ROS 2 Interface

The architecture of ROS2SmolVLA utilizes a decentralized edge-to-workstation paradigm designed to decouple real-time hardware execution from resource-intensive multi-modal inference. The communication layer relies on *lerobot-ros*, acting as a lightweight broker between the LeRobot and ROS 2 ecosystems. This abstraction enables the core VLA policies to seamlessly operate on raw joint trajectories or Cartesian velocity spaces without managing low-level industrial bus cycles directly. Figure 1 sketches the interactions between the system’s software components. At the hardware execution layer, the robot arm’s actuation modes are abstracted through standardized `ros2_control` controllers. Depending on operational requirements, the system exposes four ways of interacting with the robot: direct joint position control (`position_controllers/JointGroupPositionController`), joint trajectory interpolation (`joint_trajectory_controller`), real-time operational-space velocity tracking utilizing `MoveIt Servo`, and publishing a `TwistStamped` message for ingestion by the Cartesian Motion Controller through our Cartesian Operator. For the Robotiq Hand-E end-effector, gripper commands are processed concurrently via either a trajectory-based controller or a feedback-oriented (`Parallel`)GripperActionController. State tracking and feedback verification are

broadcast across the local ecosystem using a centralized `joint_state_broadcaster` and the Cartesian Controllers' `current_pose` topic for the current Cartesian pose of the end-effector. By utilizing Cartesian coordinates instead of direct joint control, our approach can easily be ported to different embodiments.

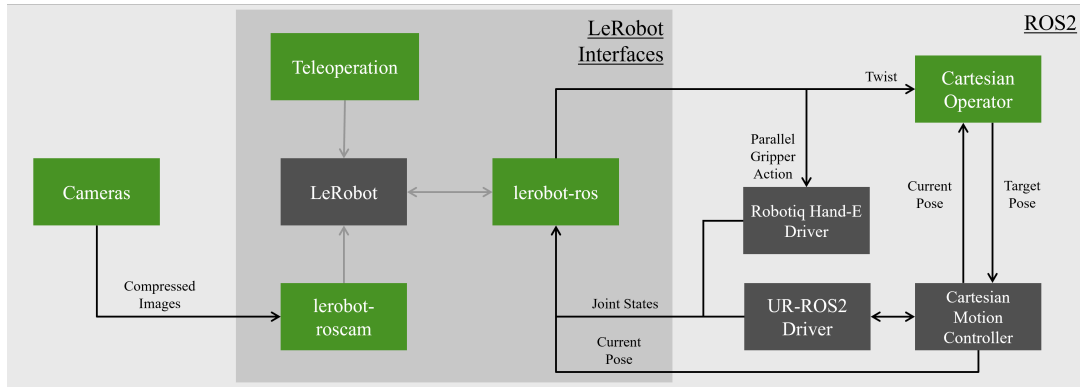


Figure 1: Architecture of ROS2SmolVLA. Self-developed software (green) and external dependencies (gray) are marked as boxes. Arrows indicate communication (dark: ROS 2 topics, light: LeRobot interface).

3.2. Multi-Modal Data Acquisition

To train and guide SmolVLA, a data collection and sensor aggregation layer is established across the physical and a virtual (Gazebo) platform. Visual state representation is driven by `lerobot-roscam`, a dedicated self-developed ROS 2 data-acquisition package designed to ingest asynchronous camera inputs. `lerobot-roscam` standardizes the ingestion pipelines for the workspace's two static cameras and the end-effector-mounted camera, by implementing the LeRobot camera interface and converting camera signals from ROS 2 messages thereof. This matches the multi-modal requirements of the base VLM model. Optionally, `lerobot-roscam` can also augment the incoming image by converting the color space, rotating the image, and cropping to custom resolutions. During operation, the LeRobot framework utilizes `lerobot-ros` for connecting to the robot, sending commands, and collecting the proprioceptive state. LeRobot then merges the high-dimensional visual tensors obtained from `lerobot-roscam` with low-dimensional robot telemetry obtained from `lerobot-ros`. Textual instructions or semantic language prompts are concurrently mapped to these multi-modal states within continuous action episodes. The target trajectories (joint velocities or Cartesian paths) and the normalized jaw openings are indexed into serialized data records matching LeRobot's standardized format. This allows the edge device to serialize datasets directly from the active industrial network.

3.3. Containerized Inference and Simulation Environment

The final layer of the architecture focuses on isolating dependencies, scaling model execution, and offering digital twin capabilities. To achieve consistent reproducibility between development machines, edge devices and production workstations, the software environment is packaged via `ur10e_vla_docker`. This containerized environment isolates the CUDA runtime, PyTorch extensions, and ROS 2 packages via Docker [32]. On the inference workstation, this containerized wrapper manages and converts the incoming state data, feeds them to the SmolVLA network, and computes the corresponding joint or velocity action targets at low latencies. Complementing the physical deployment stack, `ur10e_vla_sim` acts as an architectural verification and simulation layer. The simulation environment replicates the physics, camera configurations, and workspace layout of the physical robot cell inside Gazebo. As the underlying ROS 2 control API and topic schemas within `ur10e_vla_sim` (virtual twin) are entirely identical to the physical setup handled by `lerobot-ros` and `ur10e_vla_real` (the real twin), the containerized SmolVLA policies can be deployed interchangeably. This structural parity enables pre-validation and fine-tuning of the VLA policy within a simulated context prior to target execution on the physical UR10e system. Further, hybrid training data may be generated and combined from the simulated and real environment. While we integrated the simulation capabilities into ROS2SmolVLA, we utilized only real data in later validation. Therefore, the simulator was not fully validated.

4. Validation

Validation aims to verify the viability of SmolVLA in combination with industrial-grade hardware. We feature a simple pick-and-place scenario in which the robot is tasked to insert cubes into boxes. This scenario, albeit academic, is generalizable toward industrial environments, as pick-and-place tasks with small items are common. Within nine scenarios, we test diverse influential factors, including colors, positions, configurations, geometries, and object count.

4.1. Reference Hardware, Data, and Model Training

Our hardware setup (Figure 2) features two Microsoft Azure Kinect cameras, a webcam mounted to the end-effector, and a Robotiq Hand-E jaw gripper. The cameras' data streams are processed and concatenated on a Nvidia Jetson AGX Orin edge PC. The VLA is computed on a workstation with consumer-grade hardware (Nvidia RTX 4080, 16GB VRAM, 32GB RAM). The distributed system is connected via an unmanaged 10Gbit Ethernet switch. Our datasets contain three camera views, the joint states, and the Cartesian end effector position to maximize flexibility on input features for the model. We chose Cartesian delta values as actions, which are also included in the data. Training data was generated semi-structured via teleoperation. The robot and its end effector position were controlled with a standard Gamepad. The recorded episodes contain the full action sequences from the robot start configuration to cube insertion into the box. Between each repetition, the robot was manually moved back to the starting position. This resulted in slight variations of the robot starting configuration. To learn the spatial relationships of the workspace, a 5x5 cube position grid was established. For each cube position five repetitions with varied target box positions were recorded. In addition, episodes that contain multiple cubes with different colors were introduced. To enable recovery after erroneous grasps or dropped cubes, episodes showing error-state recovery patterns were also included in the data set. The dataset with a total of 349 episodes is available via Hugging Face [30].

The model was fine-tuned following the instructions provided in the LeRobot documentation using the pretrained base model by Shukor et al. [9]. Particularly, we set `train_expert_only=False` and `compile_model=True` which resulted in better learning performance. Otherwise, default parameters were used. While training runs were initially stopped after 50,000 steps, we noticed that training for 200,000 steps improved the success rates significantly, while being more demanding on compute. Therefore, we moved the training of the 200,000-steps-model to a GPU server with a NVIDIA L40S. Training on the server took about 25 hours. Note that the model can also be trained on the workstation, but training will take significantly longer. Our trained policy uses the Cartesian end effector pose and the three visual features as inputs. To make efficient use of the restricted visual token count per image, we cropped the top view to the lower half which contains all of the relevant work space (cf. Figure 2). The outputs of the model are `TwistStamped` commands which then get applied to the robot's end effector position.

4.2. Test Plan and Results

The test plan consisted of nine structured scenarios. We use the shortened scripts: cube position (CP), cube color (CC), box position (BP), box color (BC), box shape (BS), robot configuration (RC), in-distribution (ID), out-of-distribution (OOD), and test scenario (T). The following test cases were defined before testing:

- T1** Variation of ID CP (10 variants) and ID BP (3 variants).
- T2** Variation of OOD CP (10 variants) and OOD BP (3 variants).
- T3** Variation of ID CP (10 variants) and RC (6 variants).
- T4** Variation of ID CC (3 variants) and ID CP (10 variants).
- T5** Single OOD CC and variation of ID CP (10 variants).
- T6** Two different-colored cubes at ID CP (10 variants) with varied order (2 variants); single cube picked without indicating exact color in prompt.
- T7** Two different-colored cubes at in-distribution CP (10 variants) with varied order (2 variants); single colored cube picked according to color indicated in prompt.
- T8** Variation of OOD BC (2 variants) and ID BP (6 variants).
- T9** Variation of OOD BS (2 variants) and ID BP (6 variants).

Due to the way data was generated, there is no clear distinction in ID and OOD for RC. We divided the workplace into nine uniform zones and placed markers at each center on which BP were varied. As randomized BP were also used in few training data, there is no clear distinction in ID and OOD for BP. However, we chose similar ID BP to the training data. Figure 2 indicates the ID and OOD CP/BP. All tests used the prompt “Pick up the [CC] cube and put it in the [BC] box.”. For T6, the [CC] modifier was omitted.



Figure 2: Left: Hardware setup used in this paper. The main robot workspace is on the left half. The scene's lighting is well-controlled through blinders, diffuse top lights and flicker-less side lights. As the hardware is used in multiple projects, additional tools and markings are present. These are used as interferences for the target process. Right: ID (green) and OOD (red) CP, and BP in the robot's main workspace.

For reporting, we split the pick-and-place action into pick and place sub-tasks, i. e. if the cube was picked and kept gripped among the whole trajectory, and if the cube was placed into the box, respectively. If the robot recovered automatically, we counted the attempt as successful, e. g., if the robot dropped the cube but managed to pick it up again. Place scores are reported from total, hence, failed picks also result in failed place sub-actions. The evaluation dataset is available on Hugging Face [30]. Table 1 reports the variations of BP, CP, RC, and CC (T1-T5).

Table 1: Test cases T1 to T5 with evaluation of pick and place actions. Left: Variations of BP and CP (OOD BP between two zones indicated by two IDs). Middle: Variations of RC, tested with ID CP and BP Box 7. Right: ID and OOD variation of CC, tested with ID CP and BP Box 7.

	BP	Pick	Place		RC	Pick	Place		CC	Pick	Place
T1	1	90%	90%	T3	1	70%	70%	T4	Yellow	50%	50%
	7	100%	100%		2	80%	80%		Blue	60%	60%
	6	100%	40%		3	90%	80%		Gray	80%	80%
T2	5-4	70%	60%	T5	4	80%	80%	T5	Orange	60%	60%
	5-9	90%	70%		5	50%	50%				
	5-3	40%	20%		6	90%	90%				

In T6 picks were repeated until a cube was gripped to indicate which cube was chosen and whether CC influenced its manipulation. The model was presented a green and gray cube. Under this preliminaries, the model chose the green cube 75% of all repetitions. Place sub-actions always succeeded. For T7, we essentially repeated T6 with additionally indicating a desired CC in the prompt. We report the compliance with the color and the place rate in Table 2.

Table 2: CC comparison with color indicated in prompt.

	Cubes present	Prompt	Correct color picked	Place
T7	2	green	70%	100%
	2	gray	20%	90%
	2, CPs inverted	green	70%	100%
	2, CPs inverted	gray	30%	90%

Table 3 reports on the variations in BC and BS. In T8, we presented the model with a black and orange square box that were not part of training data. In T9, we presented the model with a blue round box and a cardboard box. Latter varies not only in shape but also introduces a completely new texture.

Table 3: OOD box colors and shapes. Left: Variation of OOD BC. Right: Variation of OOD BS.

	BC	Pick	Found box	Place		BS/BC	Pick	Found box	Place
T8	Black	100%	17%	17%	T9	Round Blue	100%	100%	100%
	Orange	100%	100%	17%		Cardboard	83%	50%	17%

Lastly, we report the **overall scores** as success of [pick, place, place after successful pick], respectively. In the ID cases (T1, T3, T4), the success rates were [78.33%, 72.50%, 92.47%]. In the OOD cases (T2, T5, T8, T9), the success rates were [76.56%, 46.88%, 61.22%]. Among all test cases the success rates were [77.72%, 63.59%, 81.69%].

4.3. Discussion

As indicated by T1 and T2, while the system performed reliably for most ID BP, specific spatial configurations led to lower success rates. For instance, in T1, while BP 1 and BP 7 yielded high success rates, the success of place sub-actions was significantly impaired at BP 6. This suggests a failure in spatial reasoning, even though in other experiments a similar box position, e. g., in T8/T9, did not necessarily result in lower success rates. We consider this to be a cause of a lack of training data coverage for this region of the workspace. During grasping, we observed that the policy successfully learned to grip the cube along the faces and to turn the end-effector appropriately to avoid gripping the edges. As expected, OOD RCs (T2) showed a broader performance degradation, with BP 5-3 failing at both the pick and place sub-actions. T3 highlighted a dependency on the initial RC. While most configurations maintained high success rates, RC 5 caused a drop. This is explained easily: RC 5 is located far outside the camera's starting envelope, limiting the wrist camera's effectiveness in early detection of the cube's location.

T4 and T5 revealed a distinct bias in our vision model by showing lower pick rates for yellow and orange compared to the gray cubes. Further, when presented two cubes without a specific prompt (T6), the system defaulted to picking the green cube. More critically, T7 demonstrated a failure in prompt adherence for non-dominant colors as the success rates significantly dropped when prompting for the gray cube. The system effectively ignored the condition and defaults to its inherent bias for the green cube. Overall, the color distinction capabilities of the trained policy are negligible despite 33% of the training episodes contain multiple colored cubes with explicit prompts to pick a certain color. The OOD BC/BS tests T8 and T9 exposed limitations in how the system interprets receptacles. BC and BS drastically alter the success of place sub-actions. The detection of the black box mostly failed. Interestingly, the orange box was consistently detected, but the place sub-action still mostly failed, indicating that the policy uses blue color in the wrist camera to decide when to drop the cube. This was manually verified by holding a blue object in front of the camera. Conversely, the round blue box achieved a perfect success rate, showing that the place sub-action generalizes well to certain novel shapes, provided the visual contrast allows for accurate boundary detection as the input resolution of the VLM is relatively low. While the success rate for the cardboard box – with significantly different shape and texture – dropped considerably, this still indicates the model's ability to generalize to the concept of a box.

Albeit these limitations, the accumulated scores indicate a general feasibility of SmoIVLA with industrial-grade hardware. The OOD picks were on par with ID tests while the success of place sub-actions were significantly lowered, particularly due to the bias towards blue as trigger for dropping the cube. Despite the results showing an expected gap between ID and OOD applications and the limitations of the model, the overall scores reflect a general ability of the model for generalization. From the validation, we derived measures to improve data acquisition and model performance. With suitable improvements, we assume that SmoIVLA will be a viable model architecture for small-scale applications in industrial environments.

4.4. Lessons Learned

While validation indicates that SmoIVLA is a viable option for the deployment of VLAs on industrial-grade hardware, it is lacking robustness among holistic deployment scenarios. Thus, we indicate “lessons learned” that leverage the foundations made with ROS2SmoIVLA and indicate useful extensions that will likely increase its robustness.

The model's low input image resolution impacts the efficiency in interpreting visual data. In initial tests, the model was unable to locate objects outside the wrist camera's view, likely due to the tokenization of the input images. After cropping the top view to the relevant workspace, locating the cube was significantly more successful. Thus, aggregating as much information as possible into a visual frame is crucial. This also applies to physical dimensions as very small objects may not be processed by the model at all. In addition, the VLM has difficulties to find objects occluded by other task-relevant objects. In our case, the success rate dropped in scenarios where the cube was occluded by the box. This is likely due to the aggressive token optimization that plays a significant role in locating target objects. In addition, the validation indicated a significant reliance of the model on colors and textures seen during training. On the one hand, this indicates that training data must feature all relevant configurations of colors and textures that are expected in the workspace. Further, just adding more examples for these is not sufficient, as a dominant sample population may induce significant biases. In addition, the validation showed that the model uses blue as a primary trigger for place actions. Shape and edges are only used as secondary triggers. Generating more diverse training data may overcome this limitation and push edges towards the primary feature, leading to more generalization on the nature of the objects instead of the explicit instances. On the other hand, this behavior could be exploited in application. Our model shows an inability to properly place objects in a black box due to an under-representation in the training data. Therefore, this could be an approach to specifically design for color-coded objects that the robot shall not interact with. This could be reinforced by specifically inserting negative conditioning cues into the training prompts.

To improve generalization and recovery behavior, failure state recovery episodes can teach the policy how to behave in edge cases. We observed improved model behavior and greater robustness by adding such episodes. However, during initial training these could bias the model towards unwanted behavior, e. g., dropping the object before firmly gripping it. So, while failure state recovery episodes reinforce robustness in large data sets, they could lead to divergent behavior in sparse data. Further, manual data generation is prone to introducing bias, e. g., using similar trajectories to reach the target reproduces them in the model. In addition, teleoperation is labor intensive. There are approaches for synthetic data generation that might be applicable to the training of VLAs. We prepared the package `ur10e_vla_sim` for this purpose. However, it was not fully validated to produce comparable models to the ones trained with real data.

5. Conclusion

In this paper, we presented ROS2SmolVLA, a framework for integrating Hugging Face's SmolVLA with industrial-grade lightweight robots via ROS 2. ROS2SmolVLA is composed of comprehensive components for containerization, wrapping SmolVLA, enabling synchronized vision inputs of multiple heterogeneous camera devices, and enabling actuation on the industrial-grade robot hardware and its digital twin via simulation in Gazebo. We trained our model for a pick-and-place task with 349 episodes and performed inference on compute-restricted hardware. The validation showed a general viability of the approach albeit success rates are currently too low for real-world deployment. This is mainly due to the still lack of training data. Code, data, and model parameters are available via a Github webpage (see Section 3), Github repositories (i. a., [31]), and Hugging Face [30]. From the validation and our experiences with SmolVLA, we derived lessons learned that shall guide adopters to overcome the limitations we currently encounter. In future work depth information of the cameras could be leveraged. This was considered during the system design phase but was ultimately denied as depth information is currently not supported by the LeRobot dataset format. This could be a convenient way to add additional modalities that might be able to help with spatial reasoning. In addition, the influence of the recovery episodes during first training vs. later finetuning could be investigated.

Acknowledgements

We like to thank Van Pham Nguyen for her support in data acquisition and the Bavarian Hightech-Agenda for funding this work within the AI Production Network Augsburg. Gemini 3.5/3.7 Flash was used in drafting and editing.

References

- [1] Koren, Y., Heisel, U., Jovane, F., Moriwaki, T., Pritschow, G., Ulsoy, G., et al. *Reconfigurable Manufacturing Systems*. Berlin, Heidelberg: Springer Berlin Heidelberg; 2003,doi:[10.1007/978-3-642-55776-7_19](https://doi.org/10.1007/978-3-642-55776-7_19).

- [2] Kombaya Touckia, J., Hamani, N., Kermad, L.. Digital twin framework for reconfigurable manufacturing systems (rmss): design and simulation. *The International Journal of Advanced Manufacturing Technology* 2022;120. doi:[10.1007/s00170-022-09118-y](https://doi.org/10.1007/s00170-022-09118-y).
- [3] Bortolini, M., Galizia, F.G., Mora, C.. Reconfigurable manufacturing systems: Literature review and research trend. *Journal of Manufacturing Systems* 2018;49. doi:[10.1016/j.jmsy.2018.09.005](https://doi.org/10.1016/j.jmsy.2018.09.005).
- [4] Bahrpeyma, F., Reichelt, D.. A review of the applications of multi-agent reinforcement learning in smart factories. *Frontiers in Robotics and AI* 2022;Volume 9 - 2022. doi:[10.3389/frobt.2022.1027340](https://doi.org/10.3389/frobt.2022.1027340).
- [5] Xu, S., Li, Y., Li, Q.. A deep reinforcement learning method based on a transformer model for the flexible job shop scheduling problem. *Electronics* 2024;13(18). doi:[10.3390/electronics13183696](https://doi.org/10.3390/electronics13183696).
- [6] Li, Y., Zhao, H., Jiang, H., Pan, Y., Liu, Z., Wu, Z., et al. Large language models for manufacturing. *Journal of Manufacturing Systems* 2026;86. doi:[10.1016/j.jmsy.2026.02.014](https://doi.org/10.1016/j.jmsy.2026.02.014).
- [7] Kawaharazuka, K., Oh, J., Yamada, J., Posner, I., Zhu, Y.. Vision-language-action models for robotics: A review towards real-world applications. *IEEE Access* 2025;13. doi:[10.1109/ACCESS.2025.3609980](https://doi.org/10.1109/ACCESS.2025.3609980).
- [8] Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In: Tan, J., Toussaint, M., Darvish, K., editors. *Proceedings of The 7th Conference on Robot Learning*; vol. 229. 2023, [arXiv:2307.15818](https://arxiv.org/abs/2307.15818).
- [9] Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., et al. Smolvla: A vision-language-action model for affordable and efficient robotics. 2025. [arXiv:2506.01844](https://arxiv.org/abs/2506.01844).
- [10] Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., et al. Rt-1: Robotics transformer for real-world control at scale. In: *Robotics: Science and Systems*. 2023, doi:[10.15607/RSS.2023.XIX.025](https://doi.org/10.15607/RSS.2023.XIX.025).
- [11] O'Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., et al. Open x-embodiment: Robotic learning datasets and rt-x models : Open x-embodiment collaboration0. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. 2024, doi:[10.1109/ICRA57147.2024.10611477](https://doi.org/10.1109/ICRA57147.2024.10611477).
- [12] Chen, X., Djolonga, J., Padlewski, P., Mustafa, B., Changpinyo, S., Wu, J., et al. On scaling up a multilingual vision and language model. In: *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Los Alamitos, CA, USA; 2024, doi:[10.1109/CVPR52733.2024.01368](https://doi.org/10.1109/CVPR52733.2024.01368).
- [13] Driess, D., Xia, F., Sajjadi, M.S.M., Lynch, C., Chowdhery, A., Ichter, B., et al. Palm-e: an embodied multimodal language model. In: *Proceedings of the 40th International Conference on Machine Learning*. 2023, doi:[10.5555/3618408.3618748](https://doi.org/10.5555/3618408.3618748).
- [14] Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., et al. Openvla: An open-source vision-language-action model. In: *Proceedings of The 8th Conference on Robot Learning*. 2025, [arXiv:2406.09246](https://arxiv.org/abs/2406.09246).
- [15] Yang, J., Glossop, C., Bhorkar, A., Shah, D., Vuong, Q., Finn, C., et al. Pushing the limits of cross-embodiment learning for manipulation and navigation. In: *Robotics: Science and Systems* 2024. 2024, doi:[10.15607/RSS.2024.XX.093](https://doi.org/10.15607/RSS.2024.XX.093).
- [16] Octo Model Team, , Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., et al. Octo: An open-source generalist robot policy. In: *Proceedings of Robotics: Science and Systems*. Delft, Netherlands; 2024, [arXiv:2405.12213](https://arxiv.org/abs/2405.12213).
- [17] Song, W., Chen, J., Ding, P., Zhao, H., Zhao, W., Zhong, Z., et al. Pd-vla: Accelerating vision-language-action model integrated with action chunking via parallel decoding. 2026. [arXiv:2503.02310](https://arxiv.org/abs/2503.02310).
- [18] Black, K., Galliker, M.Y., Levine, S.. Real-time execution of action chunking flow policies. In: *Annual Conference on Neural Information Processing Systems (NeurIPS)*. 2025, [arXiv:2506.07339](https://arxiv.org/abs/2506.07339).
- [19] Belkhale, S., Sadigh, D.. Minivla: A better vla with a smaller footprint. 2024. URL: <https://github.com/Stanford-ILIAD/openvla-mini>.
- [20] Zheng, Y., Li, X., Gu, S., Zheng, Y., Tian, S., Li, W., et al. Pokevla: Empowering pocket-sized vision-language-action model with comprehensive world knowledge guidance. 2026. [arXiv:2604.20834](https://arxiv.org/abs/2604.20834).
- [21] Yang, M., Chen, J., Tao, J., Zhang, Y., Liu, J., Zhang, J., et al. Low-rank adaptation for foundation models: A comprehensive review. 2025. [arXiv:2501.00365](https://arxiv.org/abs/2501.00365).
- [22] Kim, D., Bae, M., Nam, U., Kim, G., Lee, S., Shim, K., et al. Adaptive capacity allocation for vision language action fine-tuning. 2026. [arXiv:2603.07404](https://arxiv.org/abs/2603.07404).
- [23] Yu, Y., Qiu, X.. Benchmarking vision-language-action models on so-101: Failure and recovery analysis. 2026. [arXiv:2606.08881](https://arxiv.org/abs/2606.08881).
- [24] Almuzairee, A., Christensen, H.I.. Squint: Fast visual reinforcement learning for sim-to-real robotics. 2026. URL: <https://arxiv.org/abs/2602.21203>. [arXiv:2602.21203](https://arxiv.org/abs/2602.21203).
- [25] Marafioti, A., Zohar, O., Farré, M., Noyan, M., Bakouch, E., Cuenca, P., et al. Smolvlm: Redefining small and efficient multimodal models. In: *Conference on Language Modeling (COLM)*. 2025, [arXiv:2504.05299](https://arxiv.org/abs/2504.05299).
- [26] Cadene, R., Alibert, S., Capuano, F., Aractingi, M., Zouitine, A., Kooijmans, P., et al. Lerobot: An open-source library for end-to-end robot learning. In: *The Fourteenth International Conference on Learning Representations*. 2026, [arXiv:2602.22818](https://arxiv.org/abs/2602.22818).
- [27] Cadene, R., Alibert, S., Soare, A., Gallouedec, Q., Zouitine, A., Palma, S., et al. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. <https://github.com/huggingface/lerobot>; 2024.
- [28] Scherzinger, S., Roennau, A., Dillmann, R.. Forward dynamics compliance control (fdcc): A new approach to cartesian compliance for robotic manipulators. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, doi:[10.1109/IROS.2017.8206325](https://doi.org/10.1109/IROS.2017.8206325).
- [29] Aleksandrowicz, M., Klimek, K., Plachno, J., bondada a , tyilm2946, . robotiq_hande_driver. 2025. URL: https://github.com/AGH-CEAI/robotiq_hande_driver.
- [30] Böckmann, N.. Ros2smolvla. 2026. URL: <https://huggingface.co/collections/una-auxme/ros2smolvla>.
- [31] Böckmann, N.. ros2smolvla.docker. 2026. URL: https://github.com/una-auxme/ros2smolvla_docker.
- [32] Miell, I., Sayers, A.H.. *Docker in Practice*. 2nd ed.; Manning Publications; 2019. ISBN 9781617294808.